

# Cyclic XOR

### Problem

Apply

$$f(A)_i = A_{i-1} \oplus A_i \oplus A_{i+1}$$

exactly

$$2^K$$

times.

Count arrays that become all zeros.

### Algorithm

Traverse string.

Find block length.

Add: `ans += len / 3;`

### Example

aaaaaa

length = 6

6/3 = 2

Answer = 2

### Key Observation

Define:

$$T = I + L + L^{-1}$$

where  $L$  is cyclic shift.

Over GF(2):

$$(a + b + c)^{2^k} = a^{2^k} + b^{2^k} + c^{2^k}$$

Thus:

$$T^{2^k} = I + L^{2^k} + L^{-2^k}$$

### Formula

For every maximal block:

$$ans + = len/3$$

### TC & SC

Time Complexity :  $O(\log N)$

### Java Code

```
import java.util.*;
import java.lang.*;
import java.io.*;

class Codechef
{
    static final long MOD = 998244353L;

    static long modPow(long a, long e) {
        long res = 1;
        a %= MOD;

        while (e > 0) {
            if ((e & 1) == 1) {
                res = (res * a) % MOD;
            }
            a = (a * a) % MOD;
            e >>= 1;
        }

        return res;
    }

    public static void main(String[] args) throws java.lang.Exception{
        Scanner sc = new Scanner(System.in);

        int T = sc.nextInt();

        while (T-- > 0) {
            long N = sc.nextLong();
            int K = sc.nextInt();

            long s = 1L << K;
            long g = gcd(N, s);

            long m = N / g;

            long d = (m % 3 == 0) ? 2 * g : 0;

            System.out.println(modPow(2, d * K));
        }

        static long gcd(long a, long b) {
            while (b != 0) {
                long t = a % b;
                a = b;
                b = t;
            }
            return a;
        }
    }
}
```

←

✓

Difficulty: NA

☀

Expand

🔖

Statement

Submissions

Solution

Help

Cyclic XOR

You are given a cyclic 0-indexed array  $A$  of length  $N$ .

Let us define  $f(A)$  as another array produced as follows:

- $f(A)_i = A_{i-1} \oplus A_i \oplus A_{i+1}$ .

Note that  $A_{-1} = A_{N-1}$  and  $A_N = A_0$  due to cyclic array properties. Also,  $\oplus$  represents Bitwise XOR operator.

---

You are given 2 integers  $N$  and  $K$ .

Find the number of arrays such that:

- $0 \leq A_i < 2^K$
- $f^{2^K}(A)$  is the 0 array (all elements are 0). i.e. on applying  $f^{2^K}$  times on  $A$ , we get the 0 array.

Since the answer may be large, find it modulo 998244353.

Input Format

- The first line of input will contain a single integer  $T$ , denoting the number of test cases.
- The first and only line of input contains 2 integers -  $N$  and  $K$ .

Output Format

For each test case, output on a new line the number of arrays satisfying the condition modulo 998244353.

Constraints

- $1 \leq T \leq 1000$
- $3 \leq N \leq 10^9$
- $1 \leq K \leq 30$

Sample 1:

Input

Output

Java

```
8
9
10 static final long MOD = 998244353L;
11
12 static long modPow(long a, long e) {
13     long res = 1;
14     a %= MOD;
15     while (e > 0) {
16         if ((e & 1) == 1) {
17             res = (res * a) % MOD;
18         }
19         a = (a * a) % MOD;
20         e >>= 1;
21     }
22     return res;
23 }
24
25 public static void main(String[] args) throws java.lang.Exception{
26     Scanner sc = new Scanner(System.in);
27
28     int T = sc.nextInt();
29
30     while (T-- > 0) {
31         long N = sc.nextLong();
32         int K = sc.nextInt();
33
34         long s = 1L << K;
35         long g = gcd(N, s);
36
37         long m = N / g;
38
39         long d = (m % 3 == 0) ? 2 * g : 0;
40
41         System.out.println(modPow(2, d * K));
42     }
43 }
44
45 static long gcd(long a, long b) {
46     while (b != 0) {
47         long t = a % b;
48         a = b;
```

Test against Custom Input

Visualize Code

Run

Submit